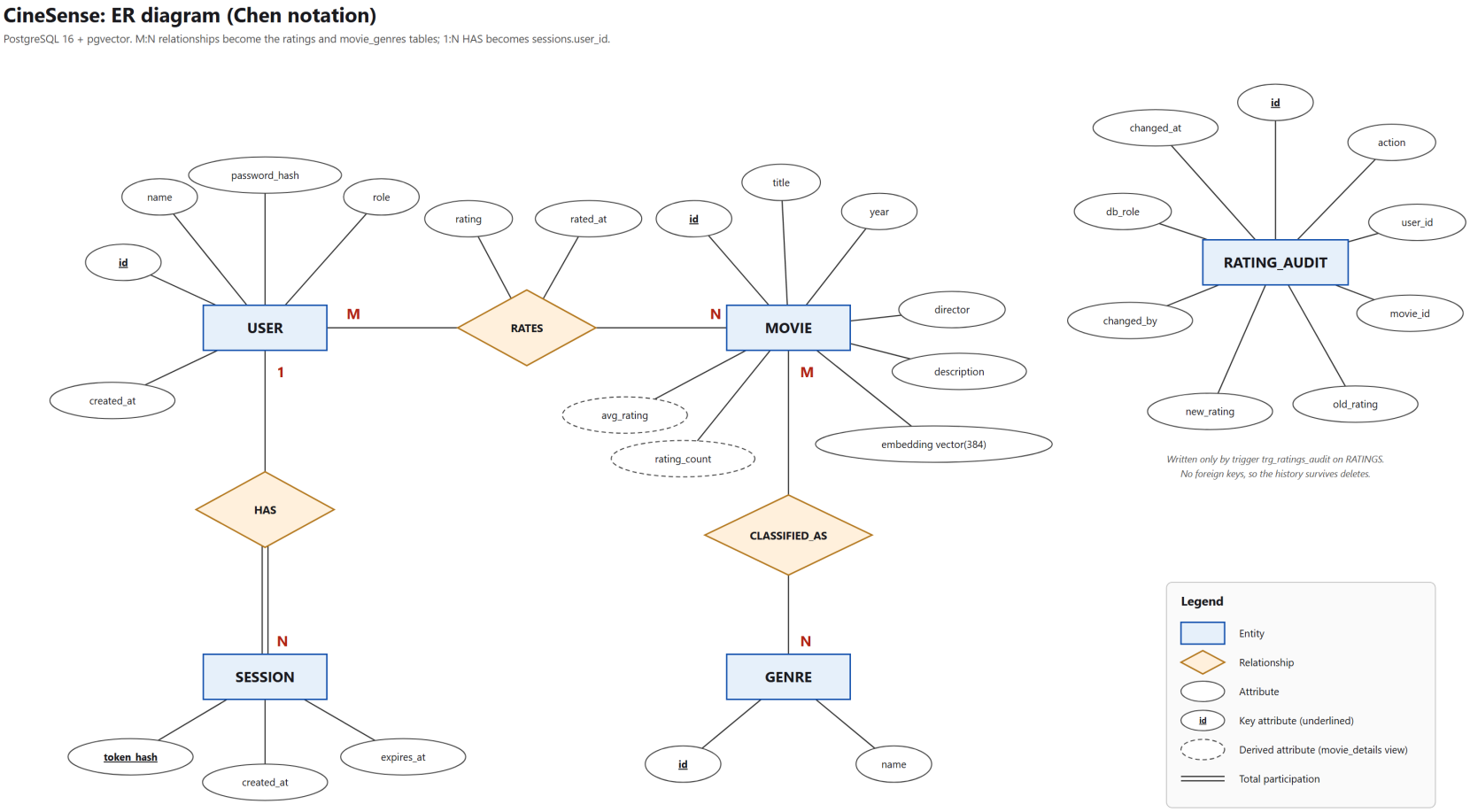
CineSense: hybrid SQL + vector movie recommender

DBMS mini project · Next.js 16 · PostgreSQL 16 + pgvector 0.8 · github.com/Prithvi-Aithal/cinesense

> **Problem statement.** Keyword or genre search only finds exact matches, so people miss movies that are similar in theme, tone or story. CineSense keeps the facts (users, movies, genres, ratings) in relational SQL tables and a 384-number embedding of each movie's meaning in a pgvector column. Recommendations combine a user's rating history (SQL) with nearest-neighbour search over the embeddings (vector), filtered by genre.

ER diagram (Chen notation)



Relational schema

```
USERS (id, name UNIQUE, password_hash, role CHECK ∈ {admin, viewer}, created_at)
MOVIES (id, title, year CHECK 1888–2100, director, description NOT NULL, embedding vector(384)) · UNIQUE (title, year)
GENRES (id, name UNIQUE)
MOVIE_GENRES (movie_id → MOVIES, genre_id → GENRES)
RATINGS (user_id → USERS, movie_id → MOVIES, rating CHECK 1–5, rated_at)
SESSIONS (token_hash, user_id → USERS, created_at, expires_at CHECK > created_at)
RATING_AUDIT (id, action CHECK ∈ {INSERT, UPDATE, DELETE}, user_id, movie_id, old_rating, new_rating, changed_by, db_role, changed_at)
VIEW MOVIE_DETAILS (id, title, year, director, description, genres[], avg_rating, rating_count)
```

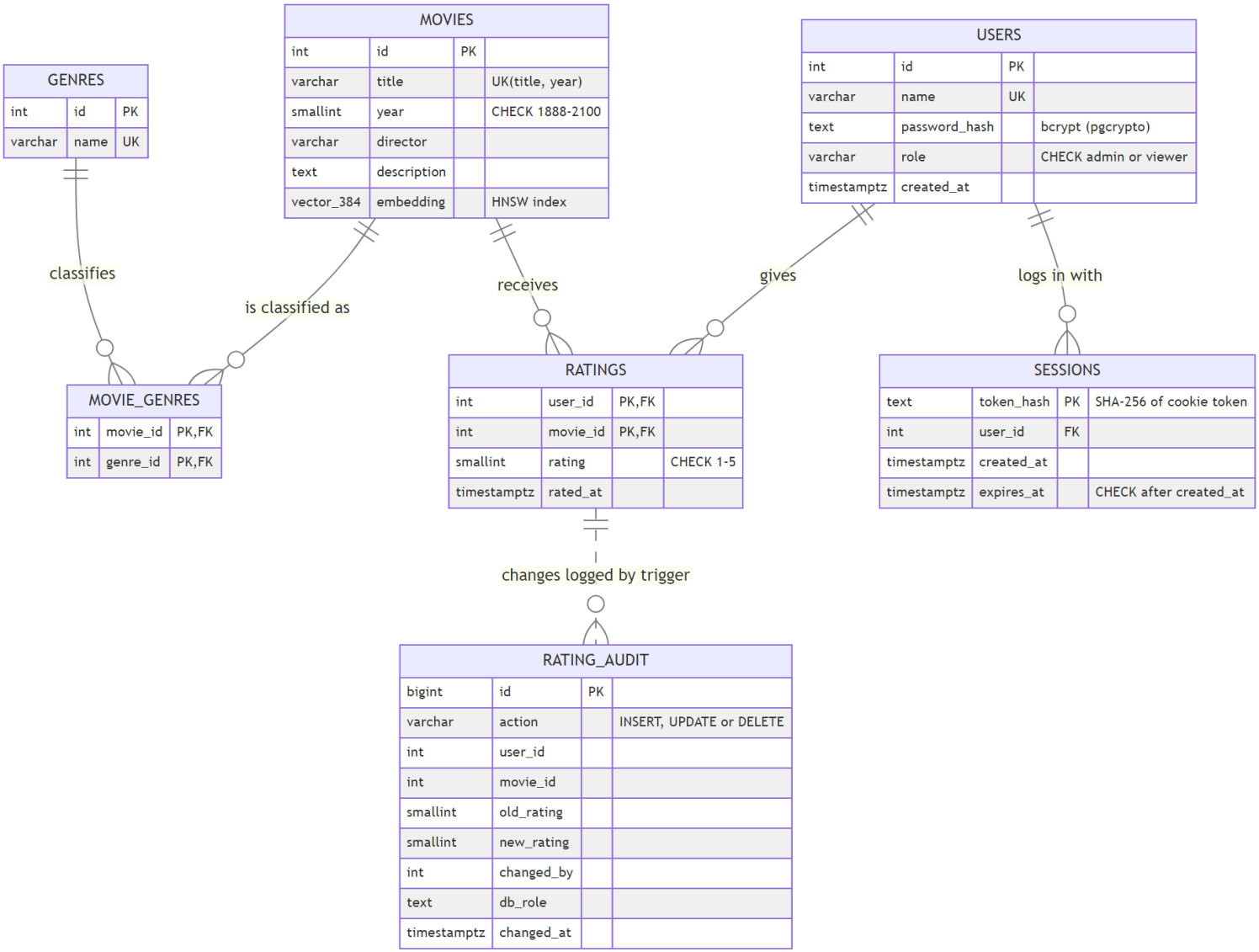
underlined = primary key · orange → = foreign key (all ON DELETE CASCADE) · RATING_AUDIT has no foreign keys on purpose (history survives deletes)

How the ER diagram became these tables

ER element	Mapping rule	Result
Strong entity	one table, key attribute → primary key	users, movies, genres, sessions, rating_audit
RATES (M:N, attributes rating, rated_at)	new table, PK = both foreign keys, relationship attributes go in it	ratings
CLASSIFIED_AS (M:N)	new table, PK = both foreign keys	movie_genres
HAS (1:N)	foreign key on the "N" side	sessions.user_id
Multivalued "genres of a movie"	separate table	genres + movie_genres
Derived attributes	not stored; computed	view movie_details

Normal form: BCNF (every determinant is a candidate key).

Relational schema diagram (crow's foot, as built by db/schema.sql)



Entities and attributes

5
strong entities
`users, movies, genres, sessions, rating_audit`

0
weak entities
every entity has its own key

3
relationships
RATES (M:N), CLASSIFIED_AS (M:N), HAS (1:N)

7
tables + **1** view
5 entity tables + 2 relationship tables

Why no weak entity? A weak entity has no key of its own and is identified through its owner plus a partial key. SESSION depends on USER for its *existence* (total participation, deleted with the user) but is *identified* by its own token_hash, so it is strong. RATINGS and MOVIE_GENRES are relationships (associative entities), not entities.

Attribute type	Meaning	In CineSense
Key	uniquely identifies an entity	users.id, movies.id, genres.id, sessions.token_hash, rating_audit.id · candidate keys: users.name, genres.name, composite (title, year)
Simple / atomic	can't be split further	every stored attribute (so all tables are in 1NF)
Composite	made of sub-parts	none
Multivalued	several values per entity	none in the final design: "genres of a movie" was multivalued and became GENRE + movie_genres
Derived	computed, not stored	movie avg_rating and rating_count (view movie_details), user's rating count, similarity (1 − cosine distance), Bayesian score · *stored but derived:* embedding (from title + genres + description)
Optional (NULL)	may be empty	movies.director, movies.embedding, rating_audit.old_rating / new_rating / changed_by
Relationship attribute	belongs to a relationship	rating, rated_at (of RATES) → stored in ratings

Constraints

Kind	Where
PRIMARY KEY	every table; composite on ratings (user_id, movie_id) and movie_genres (movie_id, genre_id)
FOREIGN KEY … ON DELETE CASCADE	ratings → users, movies · movie_genres → movies, genres · sessions → users
UNIQUE	users.name, genres.name, movies (title, year)
CHECK	rating 1–5, year 1888–2100, role admin/viewer, expires_at > created_at, audit action
NOT NULL	names, title, year, description, rating, password_hash, role, all keys
Rules in procedures / functions	last admin can't be demoted · a movie needs a genre · password ≥ 6 characters

How the database maps to the UI

Table / object	Where it appears in the UI	Create / Read / Update / Delete from the UI
users	Log in / Sign up page, header (name + role), Admin → Users & roles	C: Sign up · R: Admin → Users · U: change role · D: Admin → Delete
sessions	invisible: the login cookie	C: Log in · D: Log out (or user deleted)
movies (+ embedding)	Browse cards, Movie page, Admin → Movies & vectors	C: Add movie · R: Browse / search · U: Edit, Re-embed · D: Delete
genres, movie_genres	genre pills, genre chips on cards, admin form	created with a movie, changed by Edit, removed by Delete
ratings	stars on Browse / Movie page, My Ratings, For You	C: click stars · U: click other stars · D: My Ratings → Remove
rating_audit	Admin → Activity log	written only by the trigger; read-only in the UI
view movie_details	every movie card: genres, average rating, count	read-only (derived)
similarity_trail() (recursive CTE)	Movie page → Similarity trail	read
vector search	Browse → Meaning, More like this, For You	read
Query Lab (lib/lab.ts)	/lab (admin): joins, nested, correlated, recursive, triggers, RBAC with EXPLAIN	read-only

Request flow (Next.js)

```
Browser page (app/*/page.tsx, React) → fetch /api/… → app/api/…/route.ts (server) → lib/http.ts route(): checks the session → lib/db.ts withSession():
BEGIN · SET LOCAL ROLE cinesense_viewer/admin · SQL from lib/queries.ts · COMMIT → PostgreSQL (triggers fire here) → JSON → React re-renders
```

Triggers (db/functions.sql)

Trigger	Fires	Effect
trg_ratings_audit	AFTER INSERT / UPDATE OF rating / DELETE on ratings	writes a row to rating_audit (old, new, who, DB role)
trg_ratings_touch	BEFORE UPDATE OF rating, when the value changes	sets rated_at = now()
trg_movies_stale_embedding	BEFORE UPDATE OF title, description on movies	clears the vector if the text changed without a new one

Roles: cinesense_anon (visitor) ⊂ cinesense_viewer ⊂ cinesense_admin; the app logs in as cinesense_app (no privileges of its own) and switches role per request. Demo logins: admin / admin123 · Aarav / viewer123.